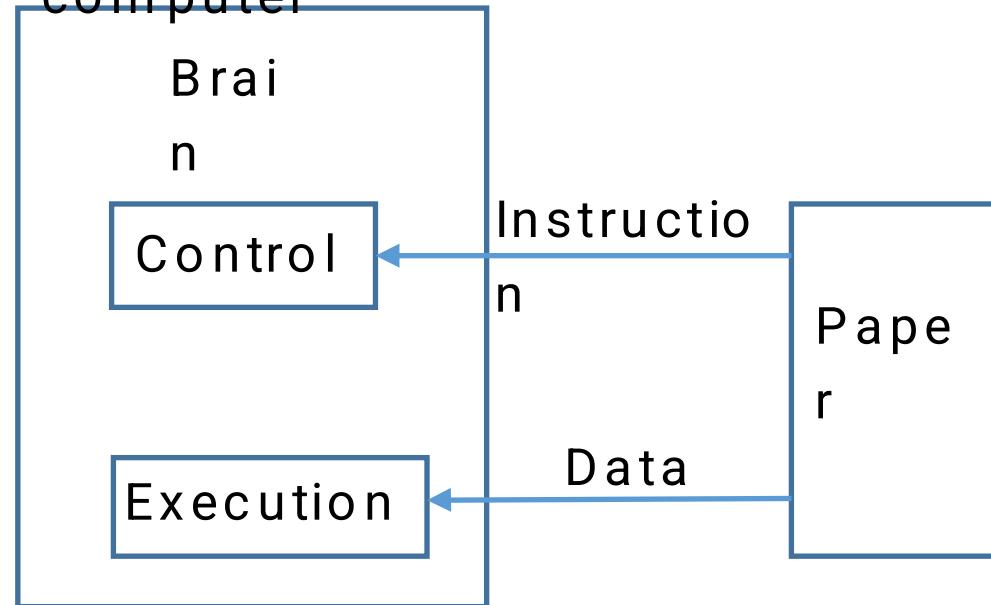


Computer organization refers to the way in which the components of a computer system are organized and interconnected to perform specific tasks. One of the most fundamental aspects of computer organization is the set of basic computer instructions that the system can execute.

Nature of computer



Brain: Computational process include two major functions

Control: Interprets instructions and ensures that the instructions are executed in proper sequence

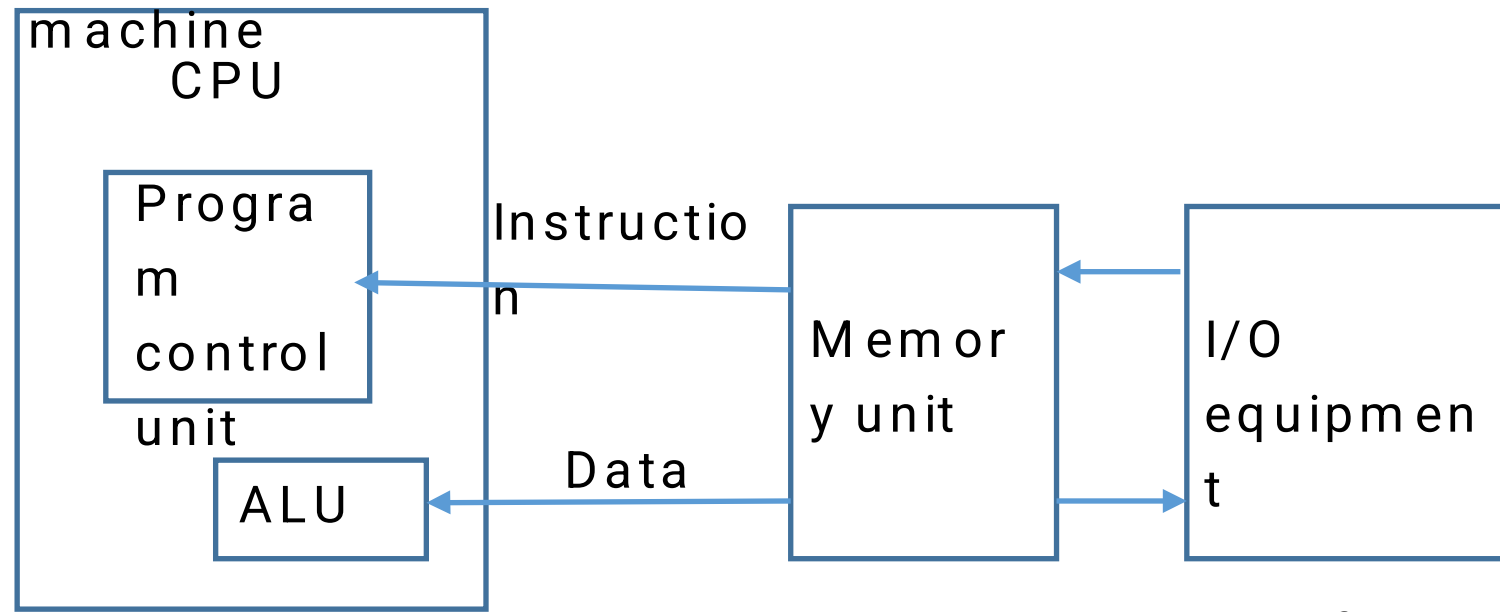
Execution: Doing specific calculations like addition, subtraction, multiplication, division

Paper: May include a list of instructions or program, intermediate or final result

Limitation: Speed of human computation is limited, complex calculations performed by hand are generally unreliable

Solution of the limitation of manual computation: Use of computing machine

Solution of the limitation of manual computation: Use of computing



CPU: Brain of human computation

Program control unit (CU): Interprets and sequences instructions

ALU: Executes instructions

Memory unit: Paper used by human calculator, stores both instructions and data

I/O equipment: Converts machine language to the language of user during machine user communication

Information is stored and processed in binary form called machine language

I/O equipment converts machine language to the language of user during machine user communication

Difference between human and machine

Prefer natural language and a wide range of symbols, decimal number

Development of computer system

1G:

Use vacuum tube

Only one programmer can access the computer at any time causes CPU idle frequently

Programs had to be written in machine language which varied from computer to computer

Machine Language: The fundamental language of computers, consisting of binary digits (0s and 1s).

Readability: Extremely difficult for humans to read, write, or debug.

Execution: Directly executed by the CPU without translation.

Dependency: Processor-specific (hardware-dependent).

2G:

Transistors replace vacuum tube in the switching circuit

Uses floating point arithmetic hardware

Machine independent high level programming language simplify programming

Introduce IO processor to supervise input output operations, thus freeing the CPU from many time consuming functions

Computer manufacturers provide system software such as compiler, subroutine, libraries etc.

Move from binary machine language to symbolic or assembly language

Assembly Language: A low-level language using mnemonics (e.g., MOV, ADD) and symbolic names (labels) for machine code instructions.

Readability: More human-readable than machine code, hence "symbolic".

Translation: Requires an **assembler** to convert it into machine code.

Dependency: Still hardware-dependent, tied to a specific CPU architecture.

3G:

Integrated circuit replaces discrete transistor circuit

Semiconductor memory replaces ferrite cores in main memory

Semiconductor Memory Ram is a Volatile memory whereas magnetic core Memory is non volatile

Semiconductor memory is faster, economical, smaller in size and lighter, but magnetic memories are slower compared to that.

Semiconductor memories consumes less power, but magnetic memory consumes more power than that.

Use microprogramming to simplify the design of CPU and increase their flexibility

Several programs run at the same time on a uniprocessor system (Multiprogramming).

Introduce pipelining and multiprocessing for concurrent or parallel processing to increase the effective speed of execution

Running of two or more programs or sequences of instructions simultaneously by a computer with more than one central processor (Multiprocessing).

Boosts performance by breaking down instruction processing into smaller, sequential stages and overlapping their execution, allowing multiple instructions to be in different stages simultaneously, thus increasing throughput (pipelining).

4G:

Uses VLSI technology

Development of VLSI has had a profound influence in lowering the cost and improving the performance of all types of computers

5G:

Multiprocessor based system

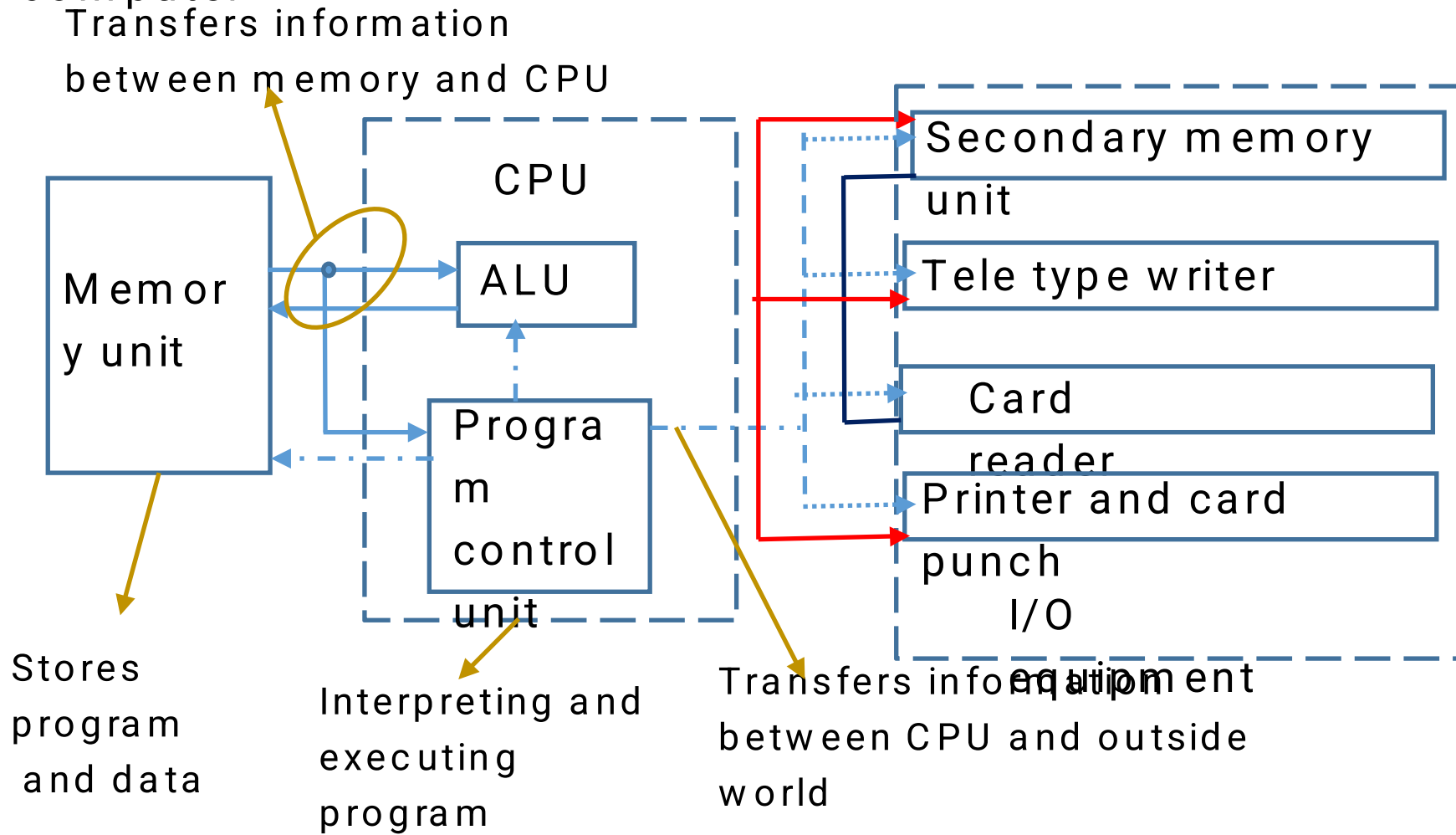
Use of AI

Use of optical fiber in circuits

Huge development of storage

Architecture of 1st generation computer

computer



Instruction: Computer programs are written in binary code or machine language

Binary string: 20-bit instruction

0011 1011 0000 0000 0111

Indicate " add the content of memory location 7 with accumulator"

Tedious to write and difficult to recognize such machine language instruction

Symbolic programming: Allows operations and addresses to be specified in symbolic mnemonic form
ADD X1: Indicate add the content of memory location X1 with accumulator

Also known as assembly language programming

Needs assembler to convert program in assembly language to machine language

First generation computer: Von Neumann machine

General purpose computers in the late 1940s and early 1950s

Control of the computer was centralized in a single CPU, all operations in the system i.e. transfer of a single word of information between IO device and main memory required direct intervention by CPU

Only one programmer could have access to the computer at any time

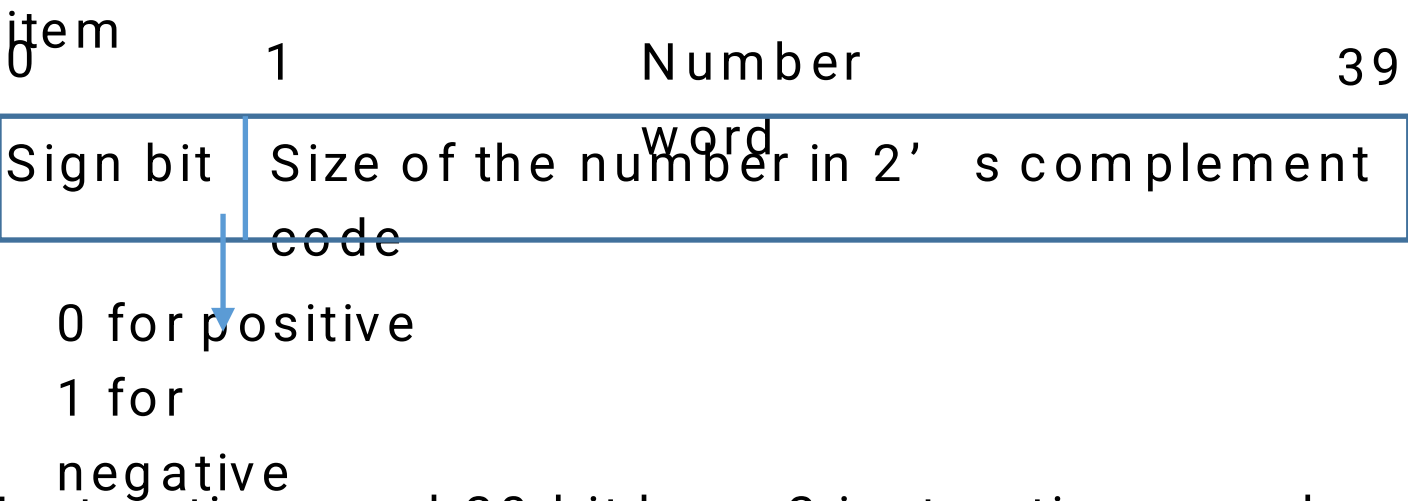
CPU becomes idle frequently due to slow I/O data transfer or control of the machine being transferred to the new programmer

Instruction format: 40-bit word i.e. the amount of information that can be transferred between main memory and CPU in one step is 40-bit

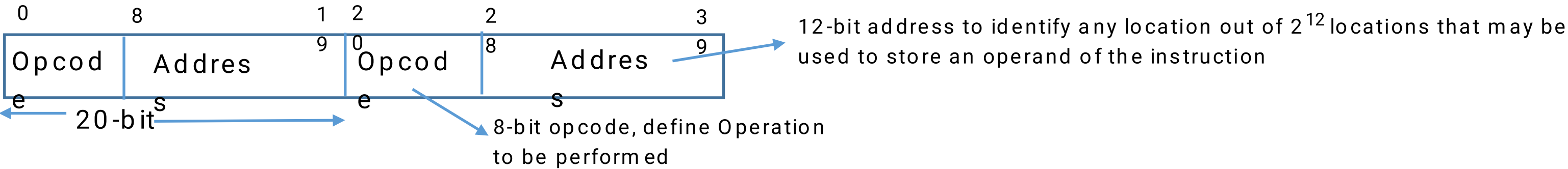
Memory capacity: $2^{12} = 4096$ words

Memory word: A memory word can be an instruction or data

Basic data



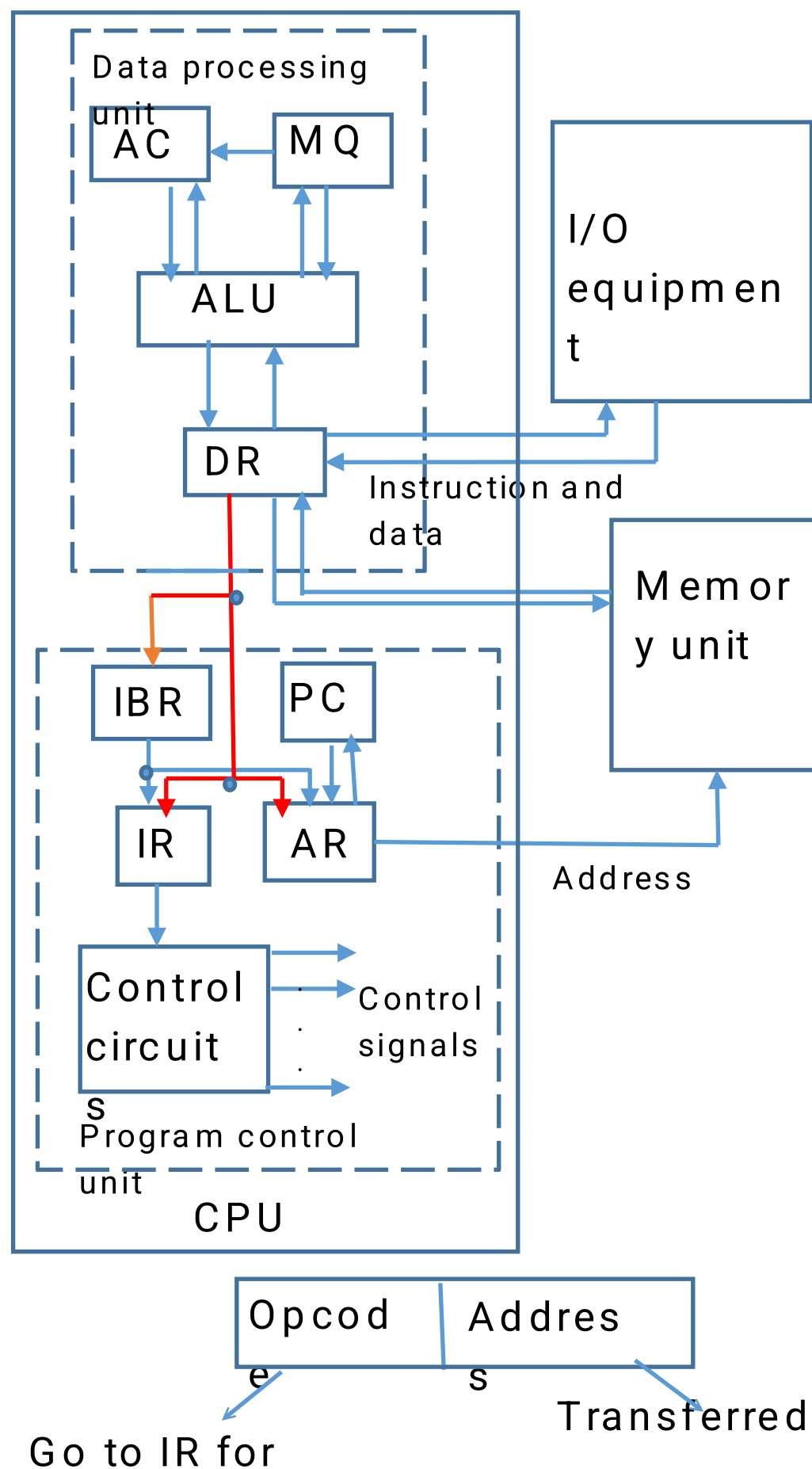
Instruction word 20-bit long, 2 instructions can be stored in each 40-bit memory location



1 memory location = 40-bit = 2 instructions

Fixed registers in the CPU are used to store operand and results. CPU register address is specified implicitly in the opcode

Instructions are stored in the main memory in the sequence in which they are executed. So address of the next pair of instruction is one plus the address of the current pair of instruction which eliminates the need of specifying next instruction address in the instruction format.



High speed register: Store instruction, data, memory address temporarily
 Accumulator (AC), Multiplier-quotient (MQ), Data register (DR), Instruction buffer register (IBR), Program counter (PC), Instruction register (IR), Address register (AR)

Arithmetic logic unit (ALU): Performs main action specified by instruction

Control circuits: Fetch instruction, decode instruction, route instruction correctly through the system, provides proper control signal for all CPU actions

Electronic clock circuit: Use to generate the basic timing signals needed to synchronize the operations of the different parts of the system

System organization

Main memory: Stores program and data

Word transfer takes place between 40-bit data register (DR) of the CPU and any location $M(X)$ with address X in M

Address X to be used is stored in a 12-bit address register (AR)

DR also stores operand during the instruction execution

Two instructions are fetched from main memory simultaneously. Instruction not to be executed immediately is stored in IBR, other instruction is the current instruction. Opcode of the current instruction is placed in IR for decoding, address field of the current instruction is transferred to the memory address register (AR)

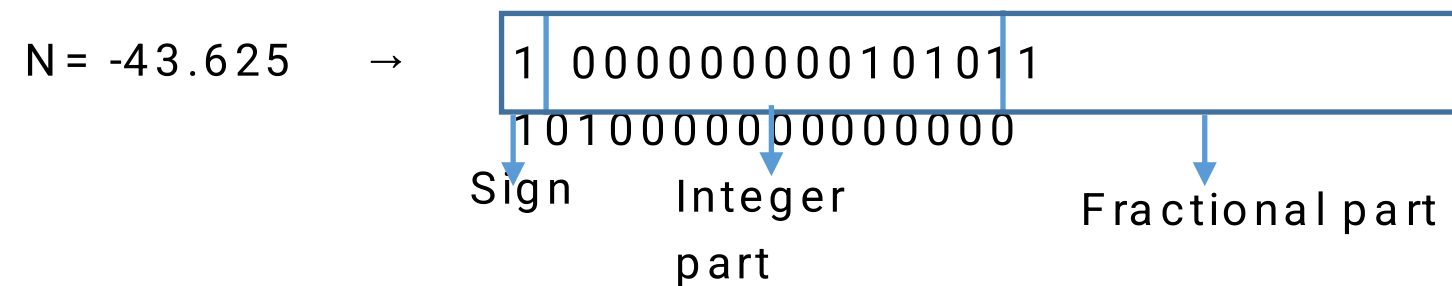
PC stores the address of the next instruction to be executed

Two major approaches to store real numbers

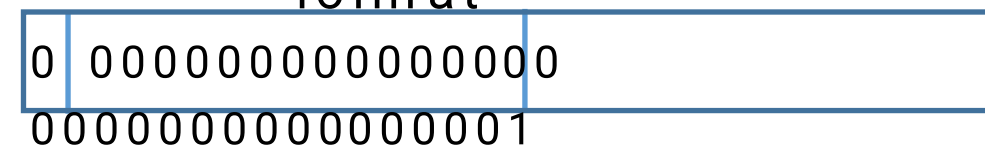
Fixed point notation has a fixed number of digits after the decimal point

Floating point notation has a varying number of digits after the decimal point

Fixed point representation of a number has fixed number of bits for integer part and fractional part
Example, 32-bit format, 1 bit for sign, 15-bit for the integer part and 16-bit for the fractional part

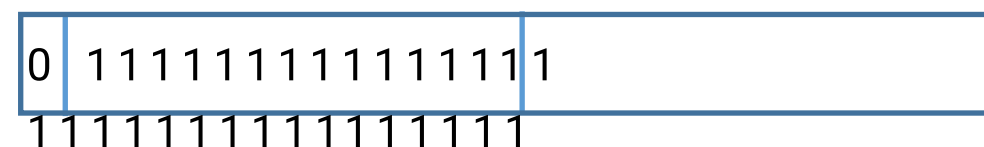


Smallest positive number in 32-bit format



$= 2^{-16} = 0.000015$
approximately

Largest positive number in 32-bit format



$= (2^{15}-1) + (1-2^{-16}) = 32768$

Disadvantage

Relatively limited range of values that they can represent

Inadequate for numerical analysis as it does not allow enough numbers and accuracy

A number whose representation exceeds 32-bit would have to be stored inexactly

Floating point representation

The range of numbers that can be represented by a fixed point number code is insufficient for many applications, particularly scientific computations where very large or very small numbers are frequently encountered.

Scientific notation permits such number to be represented using relatively few digits.

For example, it is easier to write 1.0×10^{18} (scientific notation) than 1 000 000 000 000 000 000 000 (fixed point notation)

Three numbers are associated with floating point number, format $M \times B^E$

M = mantissa, B = base, E = exponent

The precision of $M \times B^E$ is determined primarily by the number of digits used in M. The range is determined by B and E.

Floating point number representation is inherently redundant in the sense that the same number can be represented in more than one way.

For example, $1.0 \times 10^{18} = 0.1 \times 10^{19} = 1000000 \times 10^{12} = 0.000001 \times 10^{24}$

Normalization is used to specify the floating point number in a unique normal form for computer implementation

The mantissa is said to be normalized if the digit to the right of the binary point is not 0 i.e. there is no leading 0's in the magnitude part of the number.

So 0.1×10^{19} is the unique normal form for the number 1.0×10^{18} .

Unnormalized floating point number is easily normalized by shifting the mantissa to the right or left and appropriately incrementing or decrementing the exponent

Representation of 0 in floating point number system

M=0 but the exponent may have any value as $0 \times B^E = 0$ for all values of E

Representation of very small number needs a very large negative exponent, say – K. For example, 0.1×10^{-56}

If k bits are allowed for the exponent field including its sign then 2^k exponent bit patterns are available to represent signed integers, which can range either from -2^{k-1} to $2^{k-1}-1$ or from $-2^{k-1} + 1$ to 2^{k-1} . So that $K=2^{k-1}$ or $2^{k-1}-1$.

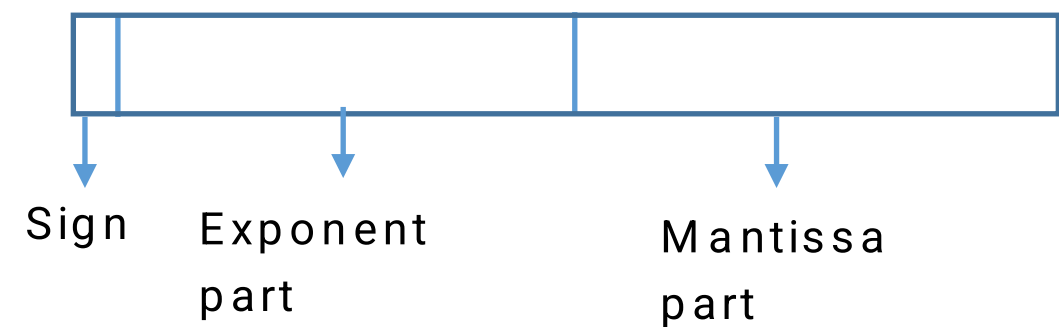
Floating point exponent be encoded in excess-K code where the exponent field contains an integer part that is the desired exponent value plus K. The quantity K is called bias and the exponent encoded in this way is called a biased exponent or characteristic.

Possible values of an 8-bit exponent with bias 127 ($= (2^{8-1}-1)$) and 128 ($= 2^{8-1}$)

Exponent bit pattern	Unsigned value E	Signed value E for K=127 ($2^{8-1}-1$)	Signed value E for K=128 (2^{8-1})
11111111	255	+128	+127
11111110	254	+127	+126
.	.	.	.
.	.	.	.
.	.	.	.
10000001	129	+2	+1
10000000	128	+1	0
01111111	127	0	-1
01111110	126	-1	-2
.	.	.	.
.	.	.	.
.	.	.	.
00000001	1	-126	-127
00000000	0	-127	-128

IEEE (Institute of Electrical and Electronics Engineers) 754: Standard format for 32-bit floating point number

32-bit floating point number: 1-bit for sign, 23-bit for mantissa and 8-bit for exponent



Mantissa part of a normalized sign magnitude number always has 1 as its most significant digit
No need to store this 1 in floating point number , since it can always be inserted by the arithmetic circuit that processes the number

In 754 format complete mantissa = 1.M, N=0.**11011111000000111111001**10111 x2³⁴ =

1.**101111100000011111001**10111 x2³³

Hidden leading bit 1 increases the precision of a normalized number by 1

Exponent is represented in 8-bit excess-127 code.

Actual exponent = E-127

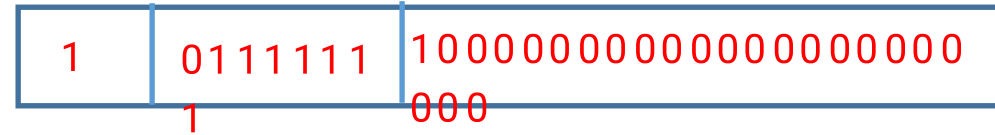
1-bit left shift (right shift) of M increases (decreases) exponent by 1

Real number represented by 32-bit floating point number in IEEE 754 standard is $N = (-1)^{S_2(E-127)}(1.M)$ provided $0 < E < 255$

i.e. $1 \leq E \leq 254$, minimum value of E is 1 and maximum value of E is 254 for valid floating point number

Example: $N = (-1)^{S_2(E-127)}(1.M)$

1) $N = -1.5_{10} = -1.1_2 = -1.1 \times 2^0$ i.e. $S=1, M=1$ and desired exponent $(E-127) = 0$



Sign bit is 1 as the number is negative

Exponent = 0, so $E-127 = 0$ and hence $E=127$

Smallest positive normalized floating point number in 32-bit format

Minimum $E = 1$ and so $E - 127 = 1 - 127 = -126$

M = 00000000000000000000000000000000

$$N = (-1)^{S_2(E-127)}(1.M) = (-1)^{0}2^{(-126)}(1.\textcolor{red}{000000000000000000000000})$$

Largest positive normalized floating point number in 32-bit format

Maximum E = 254 and so $E-127 = 254 - 127 = 127$

M = 11111111111111111111

[illegible]